



# Universidad Don Bosco

## FACULTAD DE INGENIERÍA

### ESCUELA DE COMPUTACIÓN

Desarrollo de Aplicaciones con Software Propietario  
Guía 2: Introducción a C#.Net



## I. Objetivos

Que el estudiante sea capaz de:

1. Reconocer el entorno del IDE Visual Studio .NET y comprender la estructura básica de un programa en C#.
2. Identificar los tipos de proyectos en C# .NET (consola, Windows Forms).
3. Crear un proyecto en Windows Forms con Visual Studio.
4. Implementar controles básicos en formularios (Label, TextBox, Button).

## II. Introducción Teórica

El entorno .NET es una plataforma de desarrollo integral que permite crear aplicaciones para diversos sistemas operativos y dispositivos. Originalmente, .NET Framework se centró en Windows, pero con la llegada de .NET Core (y posteriormente .NET 5+), la plataforma se volvió multiplataforma, abriendo un mundo de posibilidades para el desarrollo de escritorio, web, móvil, juegos y la nube.

Visual Studio es el entorno de desarrollo integrado (IDE) principal para C# y .NET. Ofrece potentes herramientas para escribir, depurar y desplegar código, lo que lo convierte en una herramienta indispensable para cualquier desarrollador.

En esta guía se trabajará con formularios avanzados en C#.NET utilizando Windows Forms. El objetivo principal es crear interfaces robustas e interactivas que permitan el ingreso, validación y visualización de datos de manera eficiente.

Se emplearán diversos controles como TextBox, Label, Button, CheckBox, RadioButton, PictureBox, ComboBox, ListBox y el poderoso DataGridView para trabajar con datos en forma tabular.

Desarrollaremos un formulario de registro completo en Windows Forms usando C#. Crearemos una interfaz gráfica, validar datos, manejar eventos y trabajar con imágenes.

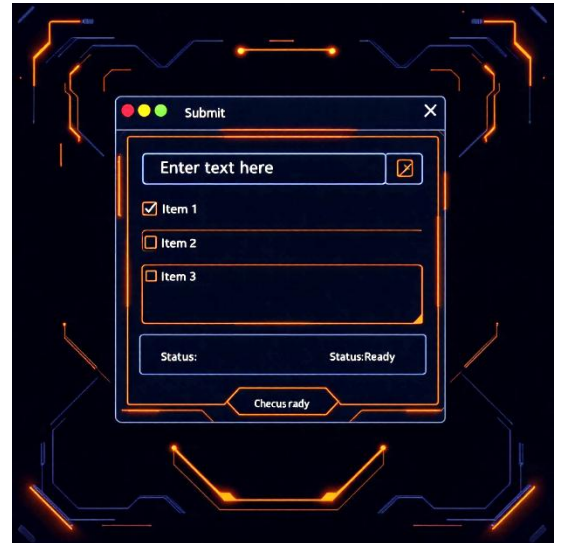
### Primeros pasos: Configuración del formulario

#### Crea un Nuevo Proyecto

Abre Visual Studio y selecciona "Crear un proyecto". Elige la plantilla "Aplicación de Windows Forms (.NET Framework)" o "Aplicación de Windows Forms" (para .NET 5+).

Los formularios de Windows Forms te permiten interactuar con el usuario mediante una variedad de controles. Aquí, exploramos algunos de los más comunes:

- **Button (Botón):** Para ejecutar acciones al hacer clic.
- **TextBox (Caja de texto):** Permite la entrada de texto por parte del usuario.
- **ListBox (Lista):** Muestra una lista de elementos seleccionables.
- **CheckBox (Casilla de verificación):** Permite al usuario seleccionar o deseleccionar una opción.
- **Label (Etiqueta):** Muestra texto estático, útil para instrucciones o títulos.



Para añadir estos controles recuerda que debes arrastrarlos desde la "Caja de Herramientas" hacia el formulario y configurar sus propiedades en la ventana "Propiedades" (como Text, Name, etc.).

### Controles Clave:

1. **PictureBox + OpenFileDialog:**
  - Para cargar/visualizar imágenes desde el sistema.
2. **TabControl:**
  - Organiza contenido en pestañas (ej: "Registro", "Reportes").
3. **CheckedListBox:**
  - Permite selección múltiple con casillas de verificación.
4. **DataGridView:**
  - Muestra datos en formato tabular.
5. **Timer:**
  - Ejecuta acciones periódicas (ej: actualizar hora).

### III. Requerimientos

| Nº | Cantidad | Descripción                           |
|----|----------|---------------------------------------|
| 1  | 1        | Guía de Laboratorio                   |
| 2  | 1        | PC con Visual Studio 2026             |
| 3  | 1        | Memoria USB o disco extraíble externo |

## IV. Procedimiento

## Ejercicio 1. Formulario de Registro de Usuario

## Paso 1

- Crear un nuevo proyecto Windows Forms con nombre 'RegistroUsuario'.
- Diseñar un formulario con nombre frmRegistro que contenga los siguientes campos:

| Tipo de Control | Nombre (Name)        | Texto               | Función                         |
|-----------------|----------------------|---------------------|---------------------------------|
| Label           | lblTitulo            | Registro de Usuario | Título principal del formulario |
| Label           | lblNombre            | Nombre Completo     | Etiqueta para el nombre         |
| TextBox         | txtNombre            |                     | Permite ingresar el nombre      |
| Label           | lblEmail             | Correo Electrónico  | Etiqueta para el correo         |
| TextBox         | txtEmail             |                     | Permite ingresar el correo      |
| Label           | lblContraseña        | Contraseña          | Etiqueta para la contraseña     |
| TextBox         | txtContraseña        |                     | Permite ingresar la contraseña  |
| Button          | btnMostrarContraseña | Mostrar             | Mostrar/Ocultar contraseña      |
| Label           | lblImagenPerfil      | Imagen de Perfil    | Etiqueta de la imagen           |
| PictureBox      | pblImagenPerfil      |                     | Muestra la imagen cargada       |
| Button          | btnCargarImagen      | Cargar Imagen       | Permite seleccionar una imagen  |
| Button          | btnRegistrar         | Registrarse         | Guarda y valida la información  |

| Tipo de Control | Nombre (Name) | Texto                | Función                               |
|-----------------|---------------|----------------------|---------------------------------------|
| Button          | btnCancelar   | Cancelar             | Limpia los campos del formulario      |
| Label           | lblUsuarios   | Usuarios Registrados | Etiqueta de los registros de usuarios |
| ListBox         | lstUsuarios   |                      | Mostrar los usuarios registrados      |

### Paso 1. Crear el Constructor del Formulario

Cuando se abre el formulario, se ejecuta el constructor. Aquí llamamos al método que configurará los controles.

```
public frmRegistro()
{
    InitializeComponent();
    ConfigurarControles();
}
```

¿Qué hace?

- InitializeComponent(); carga todos los controles creados en el diseñador.
- ConfigurarControles(); aplica configuraciones iniciales al formulario.

### Paso 2. Configurar los Controles

Crear el siguiente método dentro de la clase frmRegistro.

```
private void ConfigurarControles()
{
    txtContraseña.UseSystemPasswordChar = true;

    pbImagenPerfil.SizeMode = PictureBoxSizeMode.Zoom;
    pbImagenPerfil.BorderStyle = BorderStyle.FixedSingle;
}
```

¿Qué hace?

- Oculta la contraseña automáticamente.
- Configura el PictureBox para que la imagen se adapte correctamente.
- Agrega un borde al PictureBox.

### Paso 3. Agregar las Librerías Necesarias

Al inicio del archivo agregar:

```
using System.Text.RegularExpressions;
```

### Paso 4. Cargar una Imagen de Perfil

Crear el evento para el botón Cargar Imagen.

```
private void btnCargarImagen_Click(object sender, EventArgs e)
{
    OpenFileDialog dialogo = new OpenFileDialog();

    dialogo.Title = "Seleccionar imagen";
    dialogo.Filter =
```

```
"Imágenes|*.jpg;*.jpeg;*.png;*.bmp;*.gif|Todos los archivos|*.*";

if (dialogo.ShowDialog() == DialogResult.OK)
{
    pblImagenPerfil.Image =
        Image.FromFile(dialogo.FileName);
}
}
```

¿Qué hace?

- Abre el explorador de archivos.
- Permite seleccionar imágenes.
- Muestra la imagen dentro del PictureBox.

### Paso 5. Mostrar u Ocultar la Contraseña

Crear el evento para el botón Mostrar.

```
private void btnMostrarContraseña_Click(object sender, EventArgs e)
{
    txtContraseña.UseSystemPasswordChar =
        !txtContraseña.UseSystemPasswordChar;

    if (txtContraseña.UseSystemPasswordChar)
    {
        btnMostrarContraseña.Text = "Mostrar";
    }
    else
    {
        btnMostrarContraseña.Text = "Ocultar";
    }
}
```

¿Qué hace?

- Si la contraseña está oculta la muestra.
- Si está visible la vuelve a ocultar.

### Paso 6. Crear la Validación del Correo Electrónico

Agregar el siguiente método:

```
private bool ValidarEmail(string email)
{
    string patron =
        @"^[^@\s]+@^[^@\s]+\.[^@\s]+$";

    return Regex.IsMatch(email, patron);
}
```

Ejemplos

☒ [usuario@gmail.com](mailto:usuario@gmail.com)

☒ [alumno@udb.edu.sv](mailto:alumno@udb.edu.sv)

☐ usuario@gmail

☐ usuario.com

### Paso 7. Validar Todo el Formulario

Crear el método:

```
private bool ValidarFormulario()
{
    if (string.IsNullOrEmpty(txtNombre.Text))
    {
        MessageBox.Show(
            "Debe ingresar un nombre.");
        txtNombre.Focus();
        return false;
    }

    if (!ValidarEmail(txtEmail.Text))
    {
        MessageBox.Show(
            "Ingrese un correo válido.");
        txtEmail.Focus();
        return false;
    }

    if (txtContraseña.Text.Length < 6)
    {
        MessageBox.Show(
            "La contraseña debe tener al menos 6 caracteres.");
        txtContraseña.Focus();
        return false;
    }

    return true;
}
```

¿Qué valida?

- Nombre obligatorio.
- Correo válido.
- Contraseña mínima de 6 caracteres.

### Paso 8. Registrar al Usuario

Crear el evento del botón Registrarse.

```
private void btnRegistrar_Click(object sender, EventArgs e)
{
    if (ValidarFormulario())
    {
        string registro =
            $"Nombre: {txtNombre.Text} - " +
            $"Correo: {txtEmail.Text}";

        lstUsuarios.Items.Add(registro);
    }
}
```

```
        MessageBox.Show(
            "Usuario registrado correctamente.");

        LimpiarFormulario();
    }
}
```

¿Qué hace?

1. Ejecuta las validaciones.
2. Si todo está correcto muestra un mensaje.
3. Limpia el formulario.

### Paso 9. Limpiar el Formulario

Crear el método:

```
private void LimpiarFormulario()
{
    txtNombre.Clear();
    txtEmail.Clear();
    txtContraseña.Clear();

    pbImagenPerfil.Image = null;

    txtContraseña.UseSystemPasswordChar = true;

    btnMostrarContraseña.Text = "Mostrar";

    txtNombre.Focus();
}
```

¿Qué hace?

- Borra todos los TextBox.
- Elimina la imagen cargada.
- Oculta nuevamente la contraseña.
- Coloca el cursor en el campo Nombre.

### Paso 10. Programar el Botón Cancelar

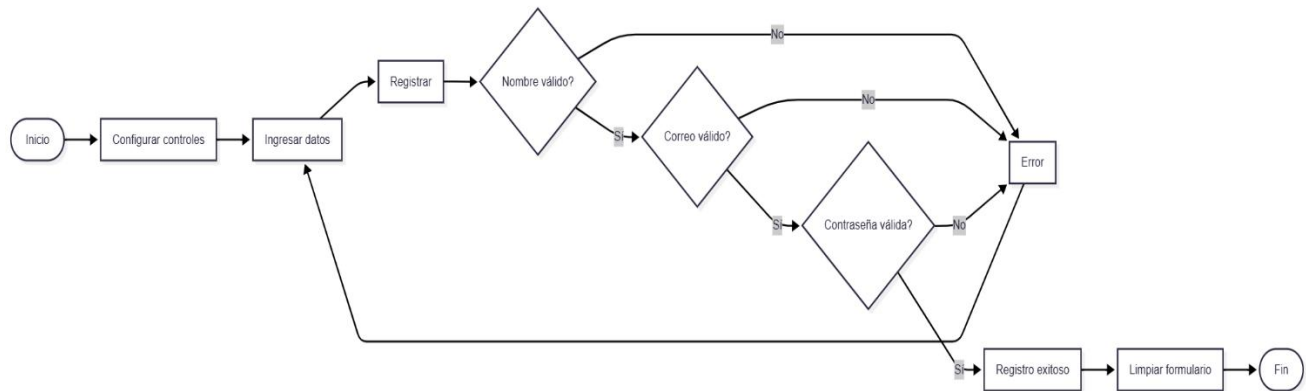
Crear el evento:

```
private void btnCancelar_Click(object sender, EventArgs e)
{
    LimpiarFormulario();
}
```

¿Qué hace?

- Borra todos los datos ingresados.
- Restablece el formulario a su estado inicial.

### Resultado Final



## V. Ejercicios complementarios

Los siguientes ejercicios se desarrollarán durante la sesión de laboratorio y tienen como objetivo reforzar los conceptos estudiados en el ejercicio guiado mediante la implementación de funcionalidades adicionales.

### Ejercicio 1. Personalización de la Interfaz

Modifique el método `ConfigurarControles()` para mejorar la apariencia visual de la aplicación.

Requisitos

- Cambiar el color de fondo de los botones.
- Modificar el color del texto de los botones.

## Desarrollo de Aplicaciones con Software Propietario

- Utilizar una fuente más llamativa para el título.
- Configurar un borde y color de fondo para el PictureBox.

Resultado esperado

El formulario debe presentar una apariencia más profesional y amigable para el usuario.

### Ejercicio 2. Mejorar la Validación de Datos

Investigue e implemente validaciones adicionales para garantizar la calidad de la información ingresada.

Requisitos

Validación del nombre

- Verificar que el usuario ingrese nombre y apellido.
- Impedir que se ingresen números o caracteres especiales.

Validación de correo

- Mantener la validación mediante expresiones regulares.
- Mostrar mensajes de error descriptivos.

Validación de contraseña

- Longitud mínima de 8 caracteres.
- Al menos una letra mayúscula.
- Al menos una letra minúscula.
- Al menos un número.

Resultado esperado

El sistema solo permitirá registrar usuarios que cumplan con todos los criterios de validación.

### Ejercicio 3. Confirmación de Contraseña

Agregue un nuevo campo denominado Confirmar Contraseña.

Requisitos

- Incorporar un nuevo Label y TextBox.
- Validar que ambas contraseñas coincidan.
- Mostrar un mensaje de error cuando los valores sean diferentes.

Resultado esperado

El registro solo debe completarse cuando ambas contraseñas sean idénticas.

### Ejercicio 4. Almacenamiento de Datos

Implemente un mecanismo para guardar y recuperar la información de los usuarios registrados mediante el uso de archivos de texto.

Requisitos

1. Utilizar StreamReader, StreamWriter o métodos de la clase File para leer y escribir información en archivos.
2. Crear un archivo CSV para almacenar la información de los usuarios.
3. Guardar cada registro en una nueva línea del archivo.
4. Implementar un botón **Guardar** para almacenar la información ingresada.
5. Implementar un botón **Cargar Datos** para leer el archivo y mostrar los registros almacenados en una tabla (DataGridView) o lista (ListBox).
6. Organizar el código en métodos independientes para separar la lógica de la interfaz gráfica y la lógica de acceso a los archivos.

#### Ejemplo de archivo CSV

Juan Pérez;juan@gmail.com;Admin123

María López;maria@gmail.com;Clave2025

Carlos Ramírez;carlos@gmail.com;Password2025

### Resultado esperado

- Los registros deben permanecer almacenados aun después de cerrar la aplicación.
- La información guardada debe poder recuperarse y visualizarse nuevamente desde el formulario.
- El usuario debe poder agregar múltiples registros al mismo archivo sin perder los datos existentes.

## VI. Análisis de resultados

### Para la próxima semana

#### Ejercicio 1. Mejorar la Experiencia de Usuario

Agregue nuevas funcionalidades a la interfaz.

##### Requisitos

- Crear un menú superior (MenuStrip).
- Agregar una opción para limpiar el formulario.
- Implementar un tema claro y un tema oscuro.
- Personalizar el icono del formulario.

##### Resultado esperado

La aplicación debe ser más intuitiva y atractiva visualmente.

#### Ejercicio 2. Seguridad Básica

Investigue mecanismos para proteger la información almacenada.

##### Requisitos

- Investigar cómo generar un hash de una contraseña.
- Analizar por qué no es recomendable almacenar contraseñas en texto plano.
- Aplicar una política de contraseñas seguras.

##### Resultado esperado

- El estudiante comprenderá conceptos básicos de seguridad aplicados al desarrollo de software.

## V. Bibliografía

1. Microsoft Learn - Documentación oficial de .NET  
<https://learn.microsoft.com/es-es/dotnet/>  
*Incluye tutoriales de C#, WinForms, y controles UI.*
2. Troelsen, A. (2023). *Pro C# 10 with .NET 6*. Apress.  
*Libro de referencia para sintaxis avanzada, patrones de diseño y buenas prácticas.*
3. Richter, J. (2023). *CLR via C#*. Microsoft Press.  
*Fundamentos del runtime de .NET y gestión de memoria.*
4. Stack Overflow - Comunidad .NET  
<https://stackoverflow.com/questions/tagged/c%23>  
*Resolución de errores comunes y mejores prácticas validadas por la comunidad.*
5. DevExpress - WinForms UI Best Practices  
<https://www.devexpress.com/winforms/>  
*Guías de diseño de interfaces y uso de controles avanzados.*

### Recursos adicionales

### Extensión de Visual Studio:

Microsoft XAML Controls Gallery para inspirar diseños modernos.

### Video-tutoriales:

Playlist de YouTube sobre WinForms (ej: "Canal de Programming with Mosh").

### Libros:

- Troelsen, A. (2022). *Pro C# 10 with .NET 6*. Apress.
- Richter, J. (2020). *CLR via C#*. Microsoft Press.